

OBJECT-ORIENTED PROGRAMMING

UNIT 1 – WELCOME TO OOP

"Programming teaches you how to write code. Object-Oriented Programming teaches you how to build software."

If your previous programming course taught you how to write programs, this course will teach you how to design software.

Throughout this semester we will use C++ to explore object-oriented software design. The language is important, but the real subject of this course is learning how to **think** about software.

ABOUT THE INSTRUCTOR

My name is Dr. Elina Benenson Opalinsky.

Before joining academia, I worked for many years in the software industry developing large-scale software systems.

Today I teach computer science and conduct research in software engineering and computer science education.

My goal is that by the end of this semester you will not simply know more C++; you will think differently about software.

COURSE ASSIGNMENTS

4 programming assignments throughout the semester

Each assignment builds upon concepts learned in class

Assignments will be submitted through **GitHub**

Start early and test your code frequently

USING AI RESPONSIBLY

- ✓ Use AI to **learn, explore, and improve**
- ✓ Ask for explanations when concepts are unclear
- ✗ Do **not** copy solutions without understanding them

You should be able to explain every line of code you submit.

BEFORE WE BEGIN

Lets take a moment to think about what you have already accomplished.

You already know how to solve computational problems using C++. You have written programs that read input, process data, make decisions, repeat operations using loops, and produce correct results.

For example, you may have written programs and/or functions that:

- Calculate the average of a list of grades.

- Search for the maximum or minimum value in an array.

- Count the frequency of characters in a string.

- Simulate a very simple menu-driven application.

- Allocate memory dynamically using new and release it with delete.

- Use struct to group related information.

These are important programming skills, and they form the foundation for everything we will do in this course.

However, writing correct code is only the first step.

As software systems grow, new questions appear:

How do we organize huge code bases?

How do teams collaborate without interfering with one another?

How can we add new features without breaking existing ones?

How do we make software easy to understand, maintain, and extend?

How do we design systems that can evolve for many years?

These are software engineering questions. They are also the questions that motivated the development of Object-Oriented Programming.

LEARNING OBJECTIVES OF THIS UNIT

- Explain why Object-Oriented Programming was developed.
- Distinguish between programming and software engineering.
- Describe the limitations of procedural programming.
- Explain modeling software using objects.
- Recognize the importance of collaboration and version control.
- Describe the four fundamental principles of Object-Oriented Programming.

PREREQUISITES

- Variables and data types (int, double, char, bool)
- Input and output (cin, cout)
- Expressions and operators
- Selection statements (if, if-else, switch)
- Loops (for, while, do-while)
- Functions
- Arrays and dynamic arrays
- Dynamic memory (new/delete)
- struct
- Basic compilation, debugging, and error handling

FROM PROGRAMS TO SOFTWARE

Programming Assignments	Professional Software
One programmer	Many developers
One deadline	Continuous development
Hundreds of lines	Thousands or millions of lines
Submit and finish	Maintain and evolve
Focus on correctness	Focus on correctness and design

PROGRAMMING VS. SOFTWARE ENGINEERING

Programming	Software Engineering
Writing code	Designing systems
Solving problems	Organizing solutions
Individual assignments	Collaborative projects
Finish and submit	Maintain for years

WHY WAS OBJECT-ORIENTED PROGRAMMING INVENTED?

Early programs were organized around procedures: read data, process data, print results. This worked well while programs were small.

As software became larger, developers encountered duplicated code, global variables, huge functions, and difficult maintenance.

Instead of organizing software around functions, Object-Oriented Programming organizes it around the important entities in the problem domain—customers, books, bank accounts, students, flights, or movies.

THE FOUR PRINCIPLES OF OBJECT-ORIENTED PROGRAMMING

Encapsulation

Protect data and expose only a controlled interface.

Abstraction

Focus on what an object does rather than how it works.

Inheritance

Build new classes from existing ones.

Polymorphism

Allow different objects to respond differently to the same operation.

Together these four principles form the foundation of modern object-oriented software design.

COURSE ROADMAP

Why OOP? struct vs. class

Classes and Objects

Encapsulation

Constructors & Destructors

Relationships

Inheritance & Polymorphism

Templates

Exceptions & STL

SOFTWARE DEVELOPMENT IS A TEAM ACTIVITY

Modern software development relies on collaboration, documentation, testing, code reviews, and version control.

Throughout this course you will submit assignments using GitHub. Besides simplifying submission, GitHub introduces you to the same workflow used by professional software engineers and helps you build a portfolio of your work.

HOW TO SUCCEED IN THIS COURSE

Write code regularly.

Experiment with every example.

Think about design before implementation.

Ask questions and learn from your mistakes.

LOOKING AHEAD

We have spent an entire unit talking about software without writing a single class. That was intentional. Before learning how Object-Oriented Programming is implemented, we first needed to understand **why** it exists.

The next unit begins with one deceptively simple question:

If struct already allows us to group related data together... why did C++ introduce class?